

# Documentação de estrutura e modelagem do Banco de dados da aplicação:

## VOLTAEXPRESS

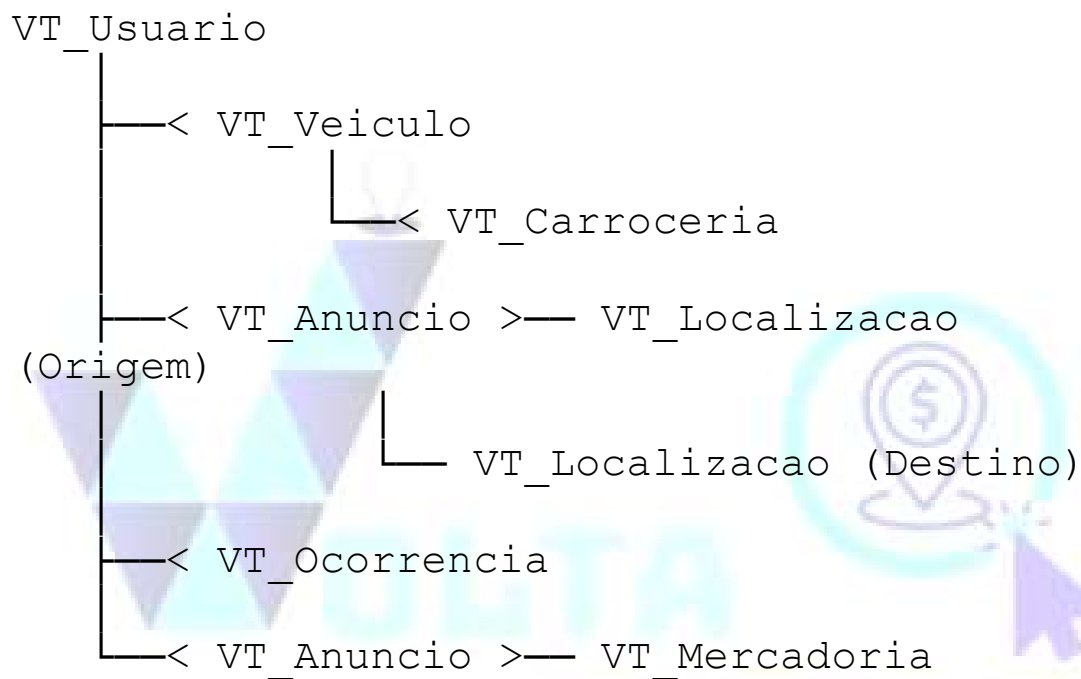
### Versão: 01



*Objetivo deste documento é orientar, direcionar e formalizar atividades da sequencia de desenvolvimento da ferramenta e suas relações. Detalhes e funções, views, procedures e regras de acessos.*

# DER – Diagrama Entidade-Relacionamento

## Visão macro dos relacionamentos



## DER VISUAL (FORMATO VISUAL / MERMAID)

erDiagram

```
VT_Usuario {
    int IdUsuario PK
    char TipoUsuario
    varchar Nome
    varchar Documento
    varchar Email
    varchar Telefone
    bit Ativo
}

VT_Veiculo {
    int IdVeiculo PK
    int IdUsuario FK
    varchar Placa
    varchar TipoVeiculo
    decimal CapacidadePesoKg
    decimal CapacidadeVolumeM3
}

VT_Carroceria {
    int IdCarroceria PK
    int IdVeiculo FK
    varchar TipoCarroceria
    bit Refrigerado
}

VT_Mercadoria {
    int IdMercadoria PK
    varchar Descricao
    varchar TipoMercadoria
    decimal PesoKg
    decimal VolumeM3
}
```

```
}
```

```
VT_Localizacao {  
    int IdLocalizacao PK  
    varchar Cidade  
    char Estado  
    decimal Latitude  
    decimal Longitude  
}
```

```
VT_Anuncio {  
    int IdAnuncio PK  
    int IdUsuario FK  
    int IdVeiculo FK  
    int IdMercadoria FK  
    int IdOrigem FK  
    int IdDestino FK  
    char TipoAnuncio  
    date DataDisponivelInicio  
    date DataDisponivelFim  
    decimal ValorSugerido  
    varchar Status  
}
```

```
VT_Ocorrencia {  
    int IdOcorrencia PK  
    int IdUsuario FK  
    varchar TipoOcorrencia  
    varchar Entidade  
    int IdEntidade  
    datetime DataOcorrencia  
}
```

```
VT_Usuario ||--o{ VT_Veiculo : possui  
VT_Veiculo ||--o{ VT_Carroceria : possui  
VT_Usuario ||--o{ VT_Anuncio : cria  
VT_Anuncio }o--|| VT_Localizacao : origem
```

VT\_Anuncio }o--|| VT\_Localizacao : destino  
VT\_Anuncio }o--|| VT\_Mercadoria : transporta  
VT\_Usuario ||--o{ VT\_Ocorrencia : gera

## Entidades e cardinalidade

### VT\_Usuario

- 1 usuário **pode ter vários veículos**
  - 1 usuário **pode ter vários anúncios**
  - 1 usuário **gera várias ocorrências**
- 

### VT\_Veiculo

- Pertence a **1 usuário**
  - Pode ter **1 ou mais carrocerias**
  - Pode ser usado em **vários anúncios**
- 

### VT\_Carroceria

- Pertence a **1 veículo**
  - Define restrições técnicas (refrigerado, tipo)
- 

### VT\_Anuncio

- Pertence a **1 usuário**
  - Pode anunciar:
    - **1 veículo** (frete de retorno)
    - **1 mercadoria** (carga disponível)
  - Sempre tem:
    - **Origem**
    - **Destino**
- 

### VT\_Localizacao

- Reutilizável
  - Evita duplicação de cidades/UF
  - Suporta geolocalização
-

## VT\_Ocorrencia

- Log central do sistema
- Pode registrar:
  - Login
  - Cadastro
  - Alteração
  - Erros
  - Ações críticas

## Procedures e Gatilhos

### Procedure 1 – Busca geral de anúncios (CORE DO APP)

```
CREATE PROCEDURE VTSP_Buscar_Anuncios
(
    @OrigemCidade    VARCHAR(100) = NULL,
    @OrigemEstado    CHAR(2) = NULL,
    @DestinoCidade    VARCHAR(100) = NULL,
    @DestinoEstado    CHAR(2) = NULL,
    @TipoVeiculo      VARCHAR(50) = NULL,
    @TipoCarroceria    VARCHAR(50) = NULL,
    @Refrigerado      BIT = NULL,
    @PesoMin          DECIMAL(10,2) = NULL,
    @PesoMax          DECIMAL(10,2) = NULL,
    @DataDisponivel   DATE = NULL,
    @TipoAnuncio      CHAR(1) = NULL
)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT *
    FROM VTW_Anuncio_Resumo
    WHERE
        (@OrigemCidade IS NULL OR OrigemCidade LIKE '%' +
        @OrigemCidade + '%')
        AND (@OrigemEstado IS NULL OR OrigemEstado = @OrigemEstado)
        AND (@DestinoCidade IS NULL OR DestinoCidade LIKE '%' +
        @DestinoCidade + '%')
        AND (@DestinoEstado IS NULL OR DestinoEstado = @DestinoEstado)
        AND (@TipoVeiculo IS NULL OR TipoVeiculo = @TipoVeiculo)
        AND (@TipoCarroceria IS NULL OR TipoCarroceria = @TipoCarroceria)
        AND (@Refrigerado IS NULL OR Refrigerado = @Refrigerado)
        AND (@TipoAnuncio IS NULL OR TipoAnuncio = @TipoAnuncio)
        AND (@PesoMin IS NULL OR PesoKg >= @PesoMin)
        AND (@PesoMax IS NULL OR PesoKg <= @PesoMax)
        AND (
            @DataDisponivel IS NULL
            OR (
                @DataDisponivel BETWEEN DataDisponivelInicio
```

```
        AND ISNULL(DataDisponivelFim, @DataDisponivel)
    )
    )
    ORDER BY DataDisponivelInicio ASC;
END;
```

### Uso no front

- Tela inicial
- Filtro lateral
- Busca avançada

---

## Procedure 2 – Matching de carga x veículo

Retornar veículos compatíveis com uma mercadoria.

```
CREATE PROCEDURE VTSP_Match_Veiculo_Mercadoria
(
    @IdMercadoria INT
)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        v.IdVeiculo,
        u.Nome AS Proprietario,
        v.TipoVeiculo,
        v.CapacidadePesoKg,
        v.CapacidadeVolumeM3,
        c.TipoCarroceria,
        c.Refrigerado
    FROM VT_Veiculo v
    JOIN VT_Usuario u ON u.IdUsuario = v.IdUsuario
    LEFT JOIN VT_Carroceria c ON c.IdVeiculo = v.IdVeiculo
    JOIN VT_Mercadoria m ON m.IdMercadoria = @IdMercadoria
    WHERE
        v.Ativo = 1
        AND (m.PesoKg IS NULL OR v.CapacidadePesoKg >= m.PesoKg)
        AND (m.VolumeM3 IS NULL OR v.CapacidadeVolumeM3 >=
m.VolumeM3);
END;
```

feito para

- Sugestão automática
  - Matching inteligente
  - Push notification futura
- 

## Procedure 3 – Busca por raio (pré-geolocalização)

**\*\*Definir se já faremos para MVP, ou deixa no roadmap**

**busca por proximidade:**

```
CREATE PROCEDURE VTSP_Busca_Por_Raio
(
    @Latitude DECIMAL(9,6),
    @Longitude DECIMAL(9,6),
    @RaioKm INT
)
AS
BEGIN
    SET NOCOUNT ON;

    SELECT
        a.IdAnuncio,
        OrigemCidade,
        OrigemEstado,
        (
            6371 * ACOS(
                COS(RADIANS(@Latitude)) *
                COS(RADIANS(lo.Latitude)) *
                COS(RADIANS(lo.Longitude) - RADIANS(@Longitude)) +
                SIN(RADIANS(@Latitude)) *
                SIN(RADIANS(lo.Latitude))
            )
        ) AS DistanciaKm
    FROM VT_Anuncio a
    JOIN VT_Localizacao lo ON lo.IdLocalizacao = a.IdOrigem
    WHERE
        lo.Latitude IS NOT NULL
        AND lo.Longitude IS NOT NULL
    HAVING
        (
            6371 * ACOS(
                COS(RADIANS(@Latitude)) *
```



```
        COS(RADIANS(lo.Latitude)) *  
        COS(RADIANS(lo.Longitude) - RADIANS(@Longitude)) +  
        SIN(RADIANS(@Latitude)) *  
        SIN(RADIANS(lo.Latitude))  
    )  
    ) <= @RaioKm  
ORDER BY DistanciaKm;  
END;
```

## INDICES E OTIMIZAÇÕES SQL

### VT\_Usuario

Usado em login, anúncios e auditoria.

```
CREATE NONCLUSTERED INDEX IDX_VT_Usuario_Email  
ON VT_Usuario (Email)  
INCLUDE (IdUsuario, Nome, TipoUsuario, Ativo);  
CREATE NONCLUSTERED INDEX IDX_VT_Usuario_Tipo_Ativo  
ON VT_Usuario (TipoUsuario, Ativo);
```

---

### VT\_Veiculo

Usado no matching.

```
CREATE NONCLUSTERED INDEX IDX_VT_Veiculo_Usuario  
ON VT_Veiculo (IdUsuario)  
INCLUDE (TipoVeiculo, CapacidadePesoKg, CapacidadeVolumeM3, Ativo);  
CREATE NONCLUSTERED INDEX IDX_VT_Veiculo_Capacidade  
ON VT_Veiculo (CapacidadePesoKg, CapacidadeVolumeM3)  
WHERE Ativo = 1;
```

---

### VT\_Carroceria

Filtro direto no app.

```
CREATE NONCLUSTERED INDEX IDX_VT_Carroceria_Tipo  
ON VT_Carroceria (TipoCarroceria, Refrigerado);
```

---

### VT\_Mercadoria

Usado no matching.

```
CREATE NONCLUSTERED INDEX IDX_VT_Mercadoria_Peso_Volume
```

```
ON VT_Mercadoria (PesoKg, VolumeM3);
```

---

## VT\_Localizacao

Muito importante para busca.

```
CREATE NONCLUSTERED INDEX IDX_VT_Localizacao_Cidade_Estado  
ON VT_Localizacao (Cidade, Estado);  
CREATE NONCLUSTERED INDEX IDX_VT_Localizacao_Geo  
ON VT_Localizacao (Latitude, Longitude)  
WHERE Latitude IS NOT NULL AND Longitude IS NOT NULL;
```

---

## VT\_Anuncio

```
CREATE NONCLUSTERED INDEX IDX_VT_Anuncio_Status_Tipo  
ON VT_Anuncio (Status, TipoAnuncio)  
INCLUDE (  
    IdUsuario,  
    IdVeiculo,  
    IdMercadoria,  
    IdOrigem,  
    IdDestino,  
    DataDisponivelInicio,  
    DataDisponivelFim,  
    ValorSugerido  
);  
CREATE NONCLUSTERED INDEX IDX_VT_Anuncio_Data  
ON VT_Anuncio (DataDisponivelInicio, DataDisponivelFim)  
WHERE Status = 'ATIVO';
```

---

## VT\_Ocorrencia (Admin / Auditoria)

```
CREATE NONCLUSTERED INDEX IDX_VT_Ocorrencia_Data  
ON VT_Ocorrencia (DataOcorrencia DESC);  
CREATE NONCLUSTERED INDEX IDX_VT_Ocorrencia_Entidade  
ON VT_Ocorrencia (Entidade, IdEntidade)
```

# API CONTRACT – APP WEB / MOBILE

Padrão **REST**, JSON

---

## Autenticação

POST /api/auth/login

### Request

```
{  
  "email": "contato@empresa.com",  
  "senha": "*****"  
}
```

### Response

```
{  
  "token": "jwt_token_aqui",  
  "usuario": {  
    "idUsuario": 12,  
    "nome": "Transportes Alpha",  
    "tipoUsuario": "E"  
  }  
}
```

## Usuários

POST /api/usuarios

```
{  
  "tipoUsuario": "C",  
  "nome": "João da Silva",  
  "documento": "12345678900",  
  "email": "joao@email.com",  
  "telefone": "279999999999",  
  "senha": "senha123"  
}
```

---

## Veículos

```
POST /api/veiculos
{
  "placa": "ABC1D23",
  "tipoVeiculo": "Carreta",
  "capacidadePesoKg": 28000,
  "capacidadeVolumeM3": 90,
  "carroceria": {
    "tipoCarroceria": "Baú",
    "refrigerado": true
  }
}
```

---

## Mercadorias

```
POST /api/mercadorias
{
  "descricao": "Carnes congeladas",
  "tipoMercadoria": "Perecível",
  "pesoKg": 18000,
  "volumeM3": 70
}
```

---

## Anúncios

```
POST /api/anuncios
{
  "tipoAnuncio": "V",
  "idVeiculo": 5,
  "origem": {
    "cidade": "Vila Velha",
    "estado": "ES"
  },
  "destino": {
    "cidade": "São Paulo",
    "estado": "SP"
  },
  "dataInicio": "2026-02-10",
  "dataFim": "2026-02-12",
  "valorSugerido": 8500,
  "observacao": "Retorno com baú refrigerado"
}
```

---

## Busca de anúncios (CORE DO APP)

GET /api/anuncios/busca

### Query Params

origemCidade=Vila  
origemEstado=ES  
destinoEstado=SP  
tipoVeiculo=Carreta  
tipoCarroceria=Baú  
refrigerado=true  
pesoMin=5000

### Response

```
[
  {
    "idAnuncio": 99,
    "tipoAnuncio": "V",
    "origem": "Vila Velha/ES",
    "destino": "São Paulo/SP",
    "tipoVeiculo": "Carreta",
    "tipoCarroceria": "Baú",
    "refrigerado": true,
    "valorSugerido": 8500
  }
]
```

## Matching inteligente

GET /api/matching/mercadoria/{idMercadoria}

```
[
  {
    "idVeiculo": 5,
    "tipoVeiculo": "Carreta",
    "capacidadePesoKg": 28000,
    "tipoCarroceria": "Baú",
    "refrigerado": true,
    "proprietario": "João da Silva"
  }
]
```

## Auditoria (Admin)

```
GET /api/ocorrencias
{
  "idOcorrencia": 1012,
  "tipo": "ALTERACAO",
  "entidade": "ANUNCIO",
  "descricao": "Alteração de valor sugerido",
  "usuario": "João da Silva",
  "data": "2026-02-04T14:22:00"
}
```

# Detalhamento Técnico / Funcional –

## 1. Objetivo deste documento

Este documento tem como objetivo **orientar desenvolvedores front-end e back-end** na compreensão, uso e evolução da aplicação de anúncios de fretes de retorno, baseada na estrutura de banco de dados modelada em **Microsoft SQL Server**.

Aqui você encontrará:

- Visão funcional da aplicação
- Explicação detalhada das tabelas
- Fluxos de dados esperados
- Uso correto de Views e Stored Procedures
- Orientações práticas para implementar o backend e integrar o frontend

Este material foi escrito com foco em **continuidade do desenvolvimento**, permitindo que qualquer desenvolvedor consiga dar sequência ao projeto com segurança.

---

## 2. Visão geral da aplicação

A plataforma funciona como um **marketplace de fretes de retorno**, onde:

- **Caminhoneiros e Empresas** se cadastram como usuários
- Usuários podem anunciar:
  - Veículos disponíveis para retorno
  - Mercadorias disponíveis para transporte
- Todos os usuários visualizam os anúncios ativos
- O sistema sugere compatibilidade entre carga e veículo

#### Perfis de usuário

| Tipo           | Código | Descrição                               |
|----------------|--------|-----------------------------------------|
| Caminhoneiro C |        | Pessoa física ou transportador autônomo |
| Empresa        | E      | Empresa contratante ou transportadora   |

---

### 3. Arquitetura lógica (resumo)

- **Banco de Dados:** SQL Server
  - **Leitura:** via Views
  - **Busca e regras:** via Stored Procedures
  - **Auditoria:** tabela centralizada de ocorrências
  - **API:** REST, retornando JSON
  - **Frontend:** Web ou Mobile (React, Vue, Flutter, etc.)
- 

### 4. Modelo de Dados – Explicação das Tabelas

#### 4.1 VT\_Usuario

Tabela central de autenticação e identidade.

##### Responsabilidade:

- Armazenar dados de login
- Definir tipo de usuário
- Controlar usuários ativos/inativos

##### Campos principais:

- TipoUsuario: C ou E
- Documento: CPF ou CNPJ
- SenhaHash: senha criptografada

##### Usada em:

- Login
  - Criação de anúncios
  - Auditoria
-

## 4.2 VT\_Veiculo

Representa veículos cadastrados por usuários.

### **Responsabilidade:**

- Informar capacidade física
- Suportar matching com mercadorias

### **Relacionamentos:**

- Pertence a um usuário
- Pode ter uma ou mais carrocerias

Um veículo pode ser reutilizado em vários anúncios.

---

## 4.3 VT\_Carroceria

Define as características operacionais do veículo.

### **Responsabilidade:**

- Restringir tipos de carga
- Permitir filtros no app

### **Exemplos:**

- Baú refrigerado
  - Sider
  - Graneleiro
- 

## 4.4 VT\_Mercadoria

Representa cargas disponíveis para transporte.

### **Responsabilidade:**

- Informar peso e volume
- Permitir matching automático

Uma mercadoria pode gerar múltiplos anúncios.

---



#### 4.5 VT\_Localizacao

Tabela reutilizável de origem e destino.

**Responsabilidade:**

- Evitar duplicação de cidades
- Permitir buscas geográficas

**Campos importantes:**

- Cidade
- Estado
- Latitude / Longitude (opcional)

---

#### 4.6 VT\_Anuncio

Tabela principal da aplicação.

**Responsabilidade:**

- Publicar fretes e cargas
- Controlar disponibilidade
- Centralizar dados para listagem

**Tipos de anúncio:**

- v: Veículo disponível
- m: Mercadoria disponível

Sempre possui:

- Origem
  - Destino
  - Status
-

#### 4.7 VT\_Ocorrencia

Tabela de auditoria e rastreabilidade.

##### **Responsabilidade:**

- Registrar ações do sistema
- Apoiar suporte e segurança

##### **Exemplos de uso:**

- Login
- Criação de anúncio
- Alteração de dados
- Erros

---

## **5. Views – Uso no Backend**

### 5.1 Conceito

As Views existem para:

- Simplificar consultas
- Garantir padrão de dados
- Melhorar performance

### 5.2 Views principais

| View                  | Uso                              |
|-----------------------|----------------------------------|
| VTW_Usuario_Ativo     | Listagem e validação de usuários |
| VTW_Veiculo_Completo  | Dados de veículos com carroceria |
| VTW_Anuncio_Resumo    | Base da home e buscas            |
| VTW_Anuncio_Busca     | Filtros rápidos                  |
| VTW_Ocorrencia_Resumo | Painel administrativo            |

✦ Backend deve consumir essas views para endpoints de leitura.

---

## 6. Stored Procedures – Regras de Negócio

### 6.1 VTSP\_Buscar\_Anuncios

**Objetivo:**

- Busca inteligente com filtros opcionais

**Usada em:**

- Tela inicial
- Busca avançada
- Filtros laterais

✦ Evita SQL dinâmico no backend.

---

### 6.2 VTSP\_Match\_Veiculo\_Mercadoria

**Objetivo:**

- Encontrar veículos compatíveis com uma carga

**Usada em:**

- Sugestões automáticas
  - Matching
  - Notificações futuras
- 

### 6.3 VTSP\_Busca\_Por\_Raio

**Objetivo:**

- Busca por proximidade geográfica

✦ Ideal para evolução futura do app.

---

## 7. Fluxos principais da aplicação

### 7.1 Cadastro de usuário

1. Front envia dados
  2. Backend valida
  3. Salva em VT\_Usuario
  4. Registra VT\_Ocorrencia
- 

### 7.2 Criação de anúncio

1. Usuário logado
  2. Backend cria localização (se não existir)
  3. Cria anúncio
  4. Status = ATIVO
  5. Registra ocorrência
- 

### 7.3 Busca de anúncios

1. Front envia filtros
  2. Backend chama VTSP\_Buscar\_Anuncios
  3. Retorna lista paginada
- 

## 8. Boas práticas para desenvolvimento

### Backend

- Sempre usar transactions
- Validar permissões por IdUsuario
- Nunca expor IDs sensíveis sem validação
- Centralizar logs na VT\_Ocorrencia

### Frontend

- Trabalhar com paginação
  - Não assumir campos obrigatórios
  - Usar filtros dinâmicos
  - Preparar UI para evolução (chat, propostas)
-

## 9. Evoluções previstas

Este modelo suporta naturalmente:

- Propostas entre usuários
- Chat em tempo real
- Avaliação pós frete
- Notificações automáticas
- Inteligência de matching

---

## 10. Conclusão

Esta documentação foi criada para garantir:

- Continuidade do projeto
- Padronização
- Facilidade de manutenção
- Escalabilidade



## Estrutura final do projeto (Express)

src/

- |— app.js           *# Configuração do Express*
- |— server.js       *# Start da aplicação*
- |— config/
  - | |— database.js   *# Conexão SQL Server*
  - | |— jwt.js       *# Config JWT*
  - | |— swagger.js   *# Swagger setup*
- |— modules/
  - | |— auth/
    - | | |— auth.controller.js
    - | | |— auth.service.js
    - | | |— auth.routes.js
  - | |— anuncios/
    - | | |— anuncio.controller.js
    - | | |— anuncio.service.js
    - | | |— anuncio.routes.js
  - | |— usuarios/
  - | |— veiculos/
  - | |— mercadorias/
- |— middlewares/
  - | |— auth.middleware.js
  - | |— error.middleware.js
- |— database/
  - | |— procedures.js
  - | |— views.js
- |— utils/
  - | |— logger.js
  - | |— response.js

## Configuração básica do Express

src/app.js

```
const express = require('express');
```

```
const swaggerUi = require('swagger-ui-express');
```

```
const swaggerSpec = require('./config/swagger');
```

```
const app = express();
```

```
app.use(express.json());
```

```
app.use('/api/docs', swaggerUi.serve, swaggerUi.setup(swaggerSpec));
```

```
app.use('/api/auth', require('./modules/auth/auth.routes'));
```

```
app.use('/api/anuncios', require('./modules/anuncios/anuncio.routes'));
```

```
app.use(require('./middlewares/error.middleware'));
```

```
module.exports = app;
```

---

```
const app = require('./app');
```

```
const PORT = process.env.PORT || 3000;
```

```
app.listen(PORT, () => {
```

```
  console.log(`API rodando na porta ${PORT}`);
```

```
});
```

# Conexão com SQL Server

**src/config/database.js**

```
const sql = require('mssql');

const config = {
  user: process.env.DB_USER,
  password: process.env.DB_PASS,
  server: process.env.DB_HOST,
  database: process.env.DB_NAME,
  options: {
    encrypt: true,
    trustServerCertificate: true
  }
};

const pool = new sql.ConnectionPool(config);
const poolConnect = pool.connect();

module.exports = { sql, pool, poolConnect };
```





## Acesso a Procedures

```
const { pool, sql } = require('../config/database');

async function buscarAnuncios(params) {
  const request = pool.request();

  Object.entries(params).forEach(([key, value]) => {
    request.input(key, value);
  });

  const result = await request.execute('VTSP_Buscar_Anuncios');
  return result.recordset;
}

module.exports = {
  buscarAnuncios
};
```

---

### Módulo de Anúncios

#### **anuncio.routes.js**

```
const router = require('express').Router();
const controller = require('./anuncio.controller');
const auth = require('../middlewares/auth.middleware');
router.get('/busca', controller.buscar);
module.exports = router;

const service = require('./anuncio.service');
```

### **anuncio.controller.js**

```
exports.buscar = async (req, res, next) => {  
  try {  
    const anuncios = await service.buscarAnuncios(req.query);  
    res.json(anuncios);  
  } catch (err) {  
    next(err);  
  }  
};
```

### **anuncio.service.js**

```
const procedures = require('../database/procedures');  
  
exports.buscarAnuncios = async (filters) => {  
  return procedures.buscarAnuncios({  
    OrigemCidade: filters.origemCidade || null,  
    OrigemEstado: filters.origemEstado || null,  
    DestinoEstado: filters.destinoEstado || null,  
    TipoVeiculo: filters.tipoVeiculo || null  
  });  
};
```

# Middleware JWT

`auth.middleware.js`

```
const jwt = require('jsonwebtoken');
```

```
module.exports = (req, res, next) => {
```

```
  const token = req.headers.authorization?.split(' ')[1];
```

```
  if (!token) return res.status(401).json({ erro: 'Token ausente' });
```

```
  try {
```

```
    req.user = jwt.verify(token, process.env.JWT_SECRET);
```

```
    next();
```

```
  } catch {
```

```
    return res.status(401).json({ erro: 'Token inválido' });
```

```
  }
```

```
};
```



# Swagger no Express

`config/swagger.js`

```
const swaggerJsdoc = require('swagger-jsdoc');
```

```
module.exports = swaggerJsdoc({  
  definition: {  
    openapi: '3.0.3',  
    info: {  
      title: 'API Fretes de Retorno',  
      version: '1.0.0'  
    },  
    components: {  
      securitySchemes: {  
        bearerAuth: {  
          type: 'http',  
          scheme: 'bearer',  
          bearerFormat: 'JWT'  
        }  
      }  
    },  
    security: [{ bearerAuth: [] }]  
  },  
  apis: ['./src/modules/**/*.routes.js']  
});
```

## Fluxo real de execução

- 1 - Front chama `/api/anuncios/busca`
  - 2 - Controller valida request
  - 3 - Service chama **stored procedure**
  - 4 - SQL Server executa lógica otimizada
  - 5 - Retorno JSON para o front
- 

## Próximos passos a partir do documento:

- Gerar **CRUD completo** (usuários, veículos, mercadorias)
- Criar **Dockerfile + docker-compose**
- Criar **testes com Jest**
- Evoluir para **roles e permissões**
- Preparar **deploy**

